FASCICULE DE COURS

DM & MACHINE LEARNING

2ème année Génie Informatique (SDIA/GLID)

Semestre 1 ♣

M. Taoufik BEN ABDALLAH

taoufik.benabdallah@iit.ens.tn

M. Radhouane Guermazi

rguermazi@gmail.com

2026-2027

Objectifs du cours

- Situer le Data Mining, le Machine Learning dans le processus ECD
- Explorer, nettoyer, transformer et préparer un jeu de données de manière reproductible
- Découvrir les règles d'association, le clustering et la réduction de dimension
- Construire et évaluer des modèles de classification et de régression
- Comparer des modèles et exploiter les méthodes d'Ensemble Learning



Organisation du cours △

Répartition approximative de la charge horaire=42h

Cours	Travaux Pratiques
22h	20h

Evaluation



DS



Projet



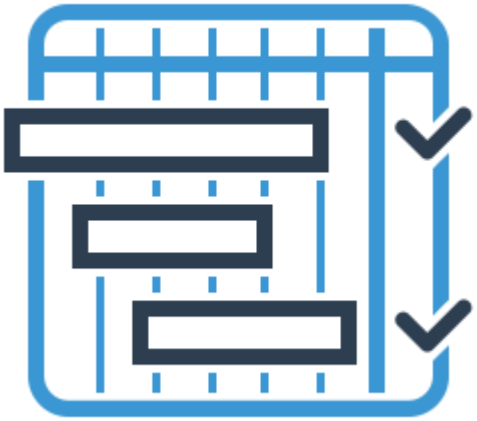
Examen

Nombre de crédit=Coefficient=3

PLAN

DU COURS

Plan du cours



1. Aperçu sur le processus KDD
2. EDA & Prétraitement de données
3. Règles d'association & découverte de motifs
4. Apprentissage non-supervisé: Clustering & Réduction de dimension
5. Apprentissage supervisé: Classification & Régression
6. Ensemble Learning & Sélection du modèle



VOLET PRATIQUE

- Calcul Scientifique avec NumPy
- Exploration des données avec pandas
- Visualisation graphique avec matplotlib & seaborn



- Prétraitement de Données, Échantillonnage, Apprentissage Automatique et Évaluation des Performances avec scikit learn

INTRODUCTION GÉNÉRALE



PANORAMA DES MÉTIERS DU DATA!

DATA ENGINEER

Mission

Collecte, transforme & fiabilise les données

Domaines

- Big Data
- ETL
- Cloud et stockage
- Data quality

Résultats

- Données fiables
- Pipelines robustes
- Données accessibles

DATA ANALYST

Mission

Analyse les données pour aider à la décision

Domaines

- Data visualization
- Business Intelligence (BI)
- KPI & Reporting

Résultats

- Tableaux de bord (Dashboards)
- Insights métier

DATA SCIENTIST

Mission

Modélise et prédit à partir des données

Domaines

- Machine Learning/ Deep Learning
- Agentic AI
- Data preprocessing pour la modélisation
- Statistique inférentielle
- Évaluation et expérimentation

Résultats

- Modèles prédictifs
- Détection d'objets& Segmentation
- Génération du contenu

De la donnée brute à la décision



L’ère des données!



☹ Masse de données souvent non-exploitée

☹ Beaucoup de données mais peu de connaissances

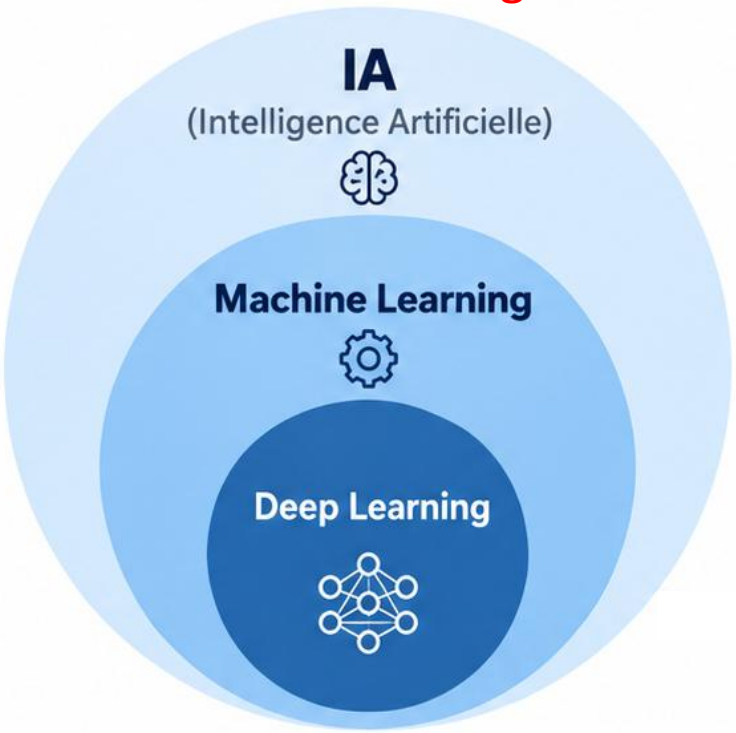
— **Comment exploiter les données ?**

Solution: Extraction des Connaissances à partir des Données (ECD) Ou **Knowledge Discovery in Databases (KDD)**

Data Mining vs Machine Learning

- ML / DL = famille de techniques : La façon d'écrire des programmes qui peuvent apprendre
- KDD = processus d' Extraction des Connaissances à partir des Données
- Data Mining = étape dans KDD
- Data Mining peut utiliser ML et parfois DL

Data Mining?



Machine Learning vs Deep Learning

Data Mining & Machine Learning

- Données tabulaires / structurées
- EDA et prétraitement explicites
- Feature engineering important
- Arbres, SVM, régression, ensembles
- Modèles souvent plus interprétables

Deep Learning

- Images, audio, texte et données massives
- Représentations apprises automatiquement
- MLP, CNN, RNN, Transformers
- Besoin plus important en données et calcul
- Très performant sur données complexes



Types d'apprentissage

Supervisé (classification& régression**)**

Données étiquetées→apprendre une fonction X → y

Non supervisé (clustering, Reduction de dimension & détection d'anomalies**)**

Pas de cible → découvrir structure, groupes, motifs ou représentations

Semi-supervisé (auto-apprentissage, co-Apprentissage**)**

Peu de données étiquetées + beaucoup de données non étiquetées

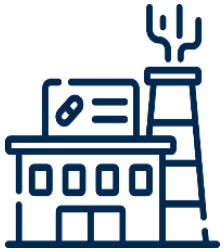
Renforcement (Q-Learning, Policy Gradient / PPO**)**

Un agent apprend par interaction à maximiser une récompense cumulée

Domaines d'application!



Recommandation



Industrie



Agriculture



Loisir



Éducation



Finance



Santé



Transport

Quelques applications en DM



Détection de fraude

Finance / banque

Détecter les transactions suspectes

Ex: fraude carte bancaire



Système de recommandation

E-commerce / média

Proposer des produits ou contenus pertinents

Ex: Amazon, Netflix



Aide au diagnostic

Santé

Aide à détecter une maladie ou un risque

Ex: Prédiction du risque de diabète



Maintenance prédictive

Industrie

Prévoir une panne avant qu'elle ne survienne

Ex: machines, capteurs IoT



Prévision des ventes

Télécom / services

Estimer les ventes futures

Ex: prévoir les ventes du mois prochain



Filtrage de spam

Messagerie / sécurité

Classer les emails indésirables

Ex. : spam / non spam

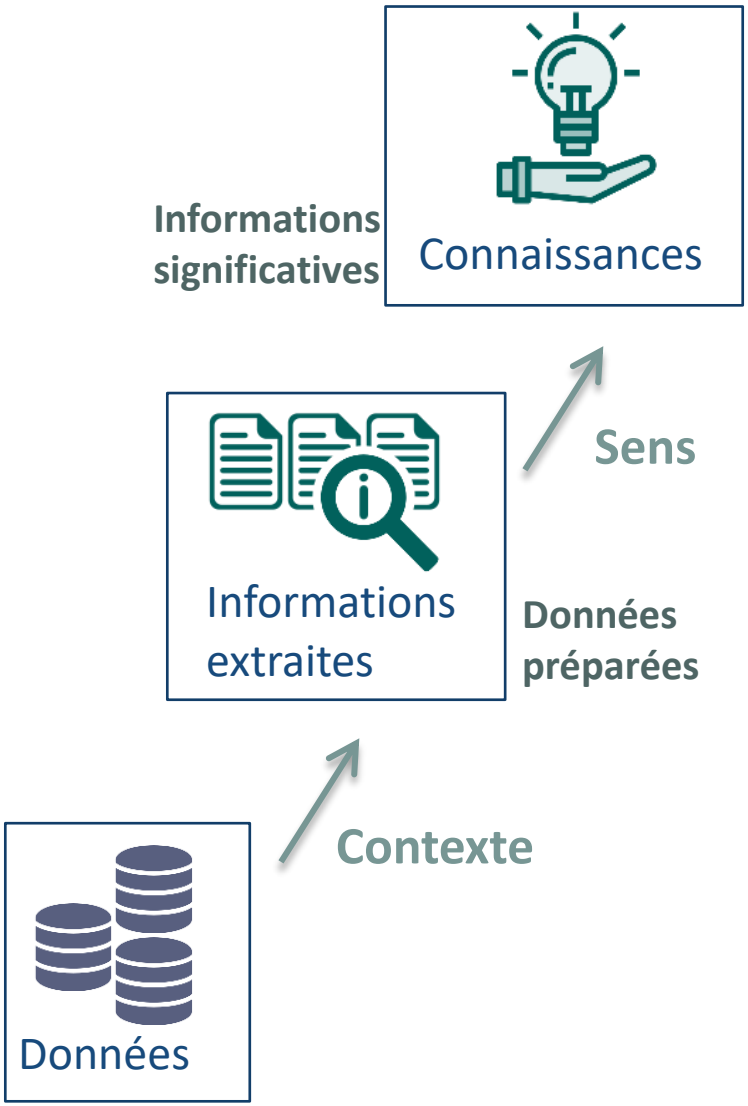




CHAPITRE 1

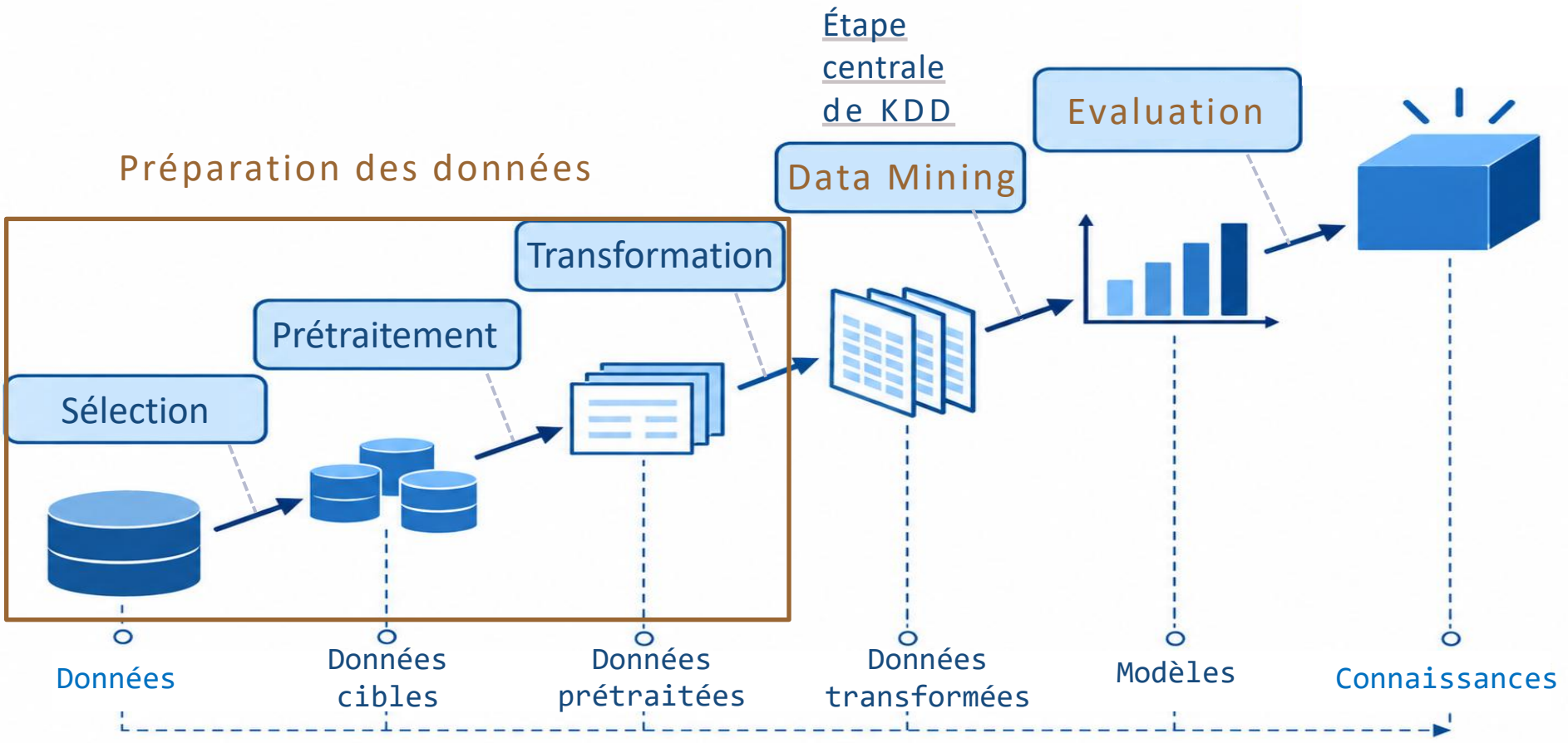
APERÇU SUR LE PROCESSUS KDD

Extraction des Connaissances à partir des Données ECD ou KDD

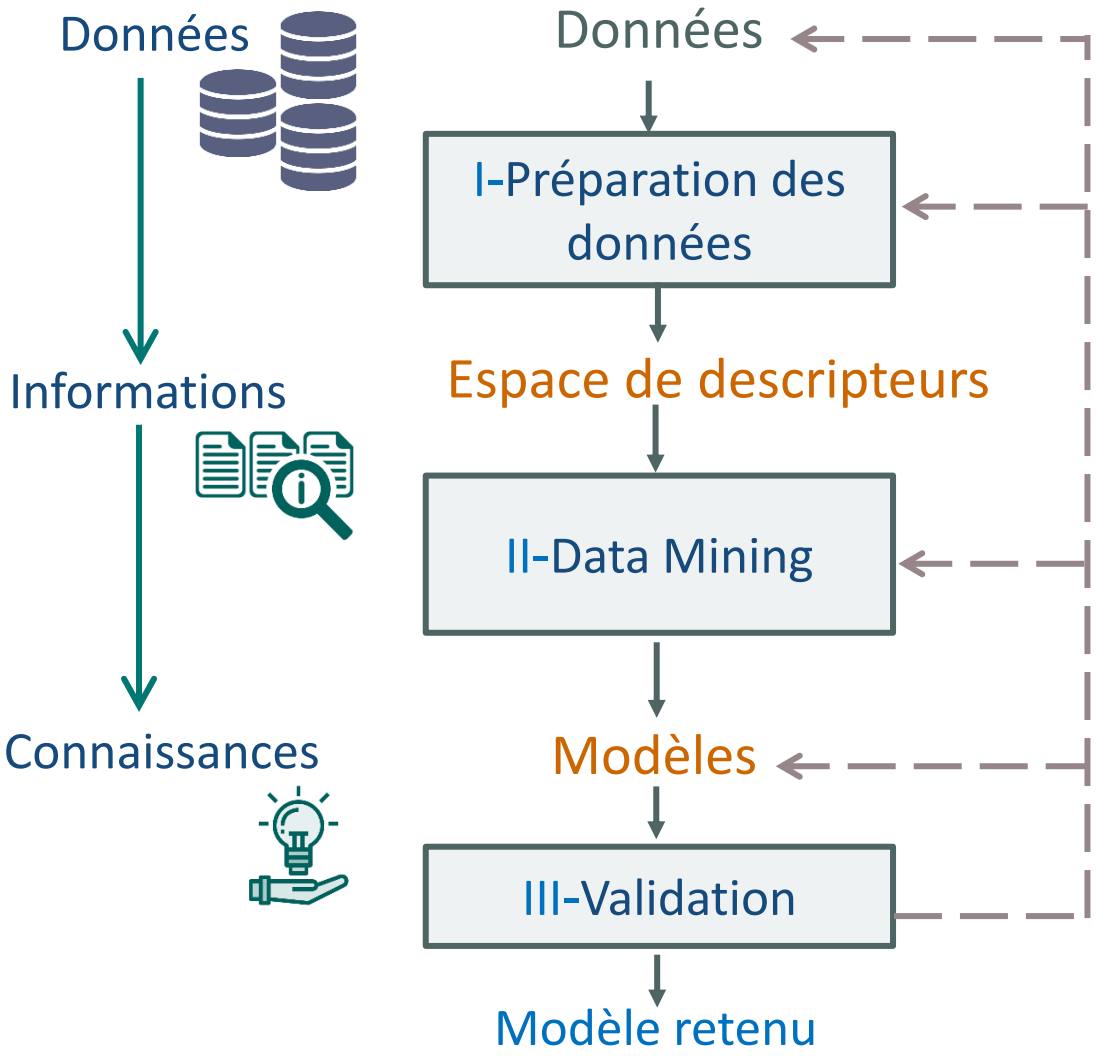




Processus KDD



Processus KDD Trois phases majeurs!



Phase I: Préparation des données

1. Détection des descripteurs

- Construction de l'espace des données qui va être exploré
- Espace des données= Jeu de donnée= *Espace de descripteurs* (Feature space)
- Collection d'observations ou objets + leurs attributs
- Une collection d'attributs décrivent un objet
- Un objet est également appelé **échantillon=entité = instance**

Client	Salaire	S. Familiale	Ville
1	Moyen	Divorcé	Tunis
2	Elevé	Célibataire	Tunis
3	Faible	Célibataire	Sfax
⋮	⋮	⋮	⋮

Observations

← **Descripteurs = Attributs = variables =caractéristiques**
= Information

1. Détection des descripteurs

- **Type de valeurs des descripteurs**

A. Discret : chaine de caractères (nominal & ordinal)

Nominal	≠	**Ordinal**
✔ Aucune relation existe entre les valeurs		✔ Les valeurs ont un ordre significatif ✔ N'est pas possible de calculer des distances
ex. Noir/ Rouge/ Blanc		**ex.** Chaud>Moyen>Froid

1. Détection des descripteurs

- **Type de valeurs des descripteurs**

B. Binaire : Seulement deux valeurs (vrai, faux / 0,1)

Symétrique	≠	**Asymétrique**
✔ Les deux résultats sont d'importance égale		✔ Les résultats n'ont pas la même importance
ex. sexe H/F		**ex.** Test médical P/N

C. Continue : Nombres entier ou réels **=valeurs quantitatives**

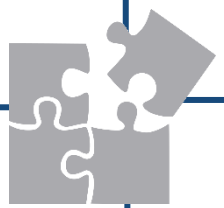
2. Prétraitement Data preprocessing

Nettoyage de données

- Remplacer les valeurs manquantes
- Supprimer les valeurs aberrantes
- Détecter les anomalies (outliers detection)

Transformation des données

- Normaliser les données



Discrétisation des données

- Convertir les attributs continus en attributs discrets

Réduction des données

- Réduire des données ou des descripteurs

PhaseII : Data Mining Tâches

- Appliquer de méthodes intelligentes pour extraire des modèles de données

Data Mining descriptif

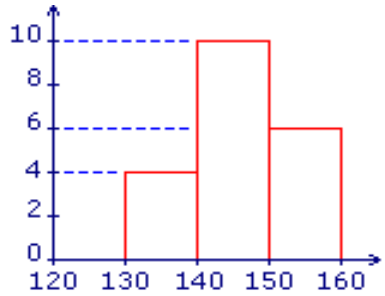
- ☑ Mettre en évidence des informations présentes mais cachées par le volume de données

Data Mining prédictif

- ☑ Extrapoler de nouvelles informations à partir des informations présentes



- **Statistique descriptive**: Résumé des données qui soit le plus intelligible → Représentation graphique





- **Découverte des règles d'association :** Découvrir des relations entre des produits (secteur de Marketing)

Apprentissage non-supervisé Clustering

→ Organisation des données en groupes

→ Les données similaires soient dans le même groupe

Client	Salaire	S. Familiale	Ville	Classe
1	Moyen	Divorcé	Tunis	
2	Elevé	Célibataire	Tunis	
3	Faible	Célibataire	Sfax	
⋮	⋮	⋮	⋮	

Pas de cible (pas de label)

Apprentissage supervisé classification/ Régression

→ Extrapoler des nouvelles informations à partir de données existantes

→ Prédire la classe de nouvelles données observées)

Client	Salaire	S. Familiale	Ville	Rembourse son crédit
1	Moyen	Divorcé	Tunis	? oui
2	Elevé	Célibataire	Tunis	? non
3	Faible	Célibataire	Sfax	? non
⋮	⋮	⋮	⋮	? ⋮

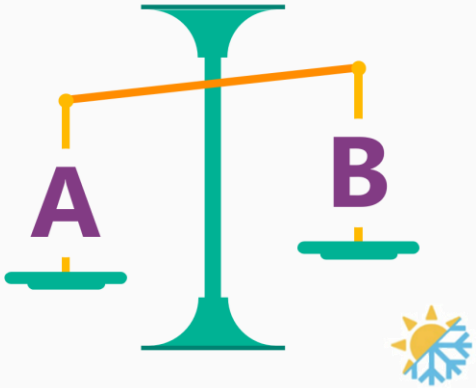
← Classe

Très couteux à avoir !!!

Apprentissage supervisé

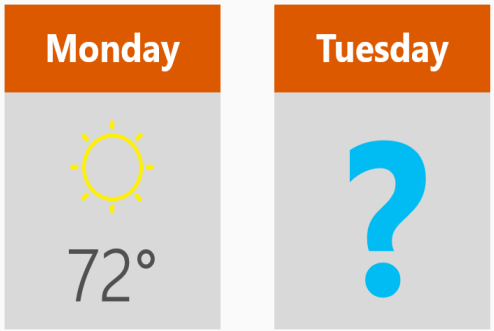
Classification : Prédire des valeurs discrètes

- Sera-t-il froid ou chaud demain? **Froid (A)/ chaud (B)**



Régression : Prédire des valeurs continues

- Quelle est la température demain?



Apprentissage supervisé

- **Techniques souvent utilisées**

 Arbres de décision	✓
 Réseau de neurones	✓
 Régression logistique/ linéaire	✓
 Support Vector Machine (SVM)	✓
 Gradient Boosting/ XGBoost/CatBoost	✓
 Random Forest	✓

Phase III: Validation

- Les modèles extraits ne peuvent être utilisés directement en toute fiabilité!

☑ **Validation=** évaluer les performances d'un modèle



- DM descriptif= Visualisation + Interprétation
- DM prédictif= Échantillonnage+Calcul d'erreur

Quelques outils de Data Mining



Librairies & Framework









Quiz

1. L'étape de préparation de données consiste principalement à (seule réponse)
   - [x] Donner un contexte aux données
   - [ ] Donner un sens aux informations
   - [ ] Évaluer et interpréter la fiabilité des données
   - [ ] Répartir les données

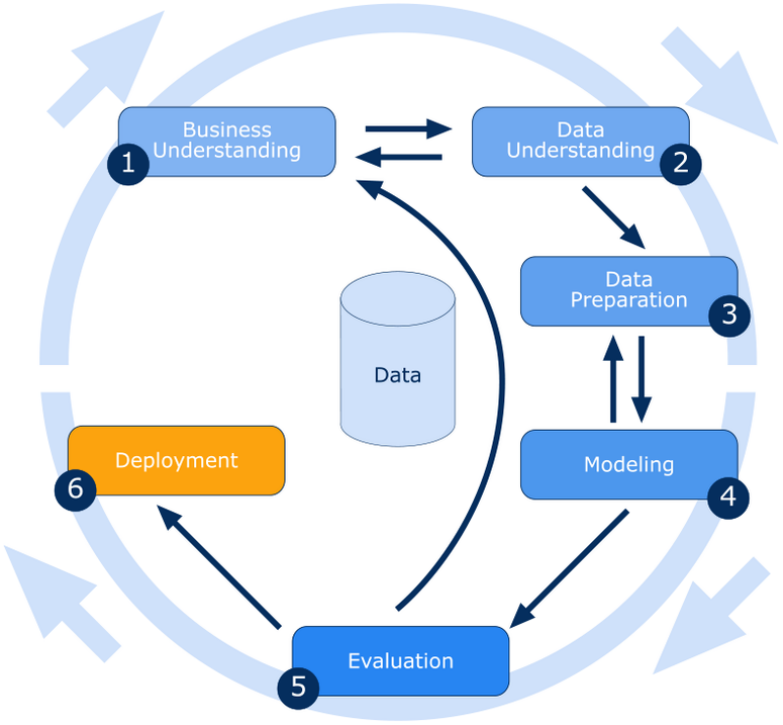
2. L'espace de descripteurs est une représentation matricielle (plusieurs réponses)
   - [ ] De connaissances
   - [x] D'informations
   - [x] Où les colonnes sont les descripteurs et les lignes sont les échantillons
   - [ ] Où les colonnes sont les échantillons et les lignes sont les descripteurs

3. L'apprentissage supervisé (seule réponse)
   - [ ] Est appliqué pour constituer des groupes d'objets homogènes et différenciés
   - [ ] Consiste à utiliser des données pour dégager des informations
   - [ ] Est appliqué principalement pour le regroupement
   - [x] Nécessite des échantillons étiquetés par un ou plusieurs classes

CRoss-Industry Standard Process for Data Mining (CRISP-DM)

- Méthodologie de référence pour structurer un projet de DM, de la compréhension du besoin métier jusqu'au déploiement

- ☑ Fournit un cadre simple, clair et réutilisable pour piloter un projet data de bout en bout



Cycle itératif
amélioration continue

1 **Compréhension du métier**
Objectifs, besoins, contraintes

2 **Compréhension des données**
Collecte initiale, qualité, exploration

3 **Préparation des données**
Nettoyage, transformation

4 **Modélisation**
Choix algorithmes, entraînement

5 **Évaluation**
Vérifier si le modèle répond au besoin

6 **Déploiement**
Mise en production, monitoring

KDD vs CRISP-DM

- ☑ Deux approches complémentaires, mais avec des objectifs différents
- KDD : Transformer des données préparées en connaissances utiles et interprétables
- CRISP-DM : Piloter un projet Data Mining depuis le besoin métier jusqu'au déploiement

Comprendre/ sélectionner les données	→	**Data Understanding**
Nettoyer et transformer	→	**Data Preparation**
Extraire un modèle / pattern	→	**Modeling**
Interpréter et vérifier	→	**Evaluation**





CHAPITRE 2

EDA & PRÉTRAITEMENT DE DONNÉES

Exemple du dataset (avec anomalies!)

Attribut date

Outliers!!

Attributs discrets

Lignes dupliquées

#	Id	Nom	Date. N	Sexe	A1	A2	A3	A4
1	111	John	31/12/1990	M	*767*	0	Ireland	Dublin
2	222	Mery	15/10/1978	F	22	0.5	Iceland	NAN
3	333	Alice	19/04/2000	F	60	300	Spain	Madrid
4	789	Alex	15/03/2000	A	35	23	Germany	Berlin
5	789	Alex	15/03/2000	A	15	23	Germany	Berlin
6	555	Peter	1983-12-01	M	NAN	10	Italy	Rome
7	777	Calvin	05/05/1995	M	32	0.13	Italy	Rome
8	888	Roxane	03/08/1948	F	44	0.8	Portugal	Lisbon
9	999	Anne	05/09/1992	F	*986*	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	24	26	Ytali	Rome

valeurs uniques

Valeurs variées

EDA & Prétraitement de données

Exploratory Data Analysis (EDA)

Analyse exploratoire des données



Analyse les données pour obtenir des informations initiales et comprendre leurs caractéristiques

Data Preprocessing

Prétraitement des données



Prépare et transforme les données afin qu'elles soient prêtes pour l'entraînement des modèles

- EDA & Prétraitement de données sont *complémentaires*
- EDA permet de mieux comprendre les données, ce qui peut orienter les décisions à prendre lors du prétraitement des données

EDA & Prétraitement de données (//)



Exploratory Data Analysis (EDA)

Analyse exploratoire des données

- **Exploration initiale :** Identification des variables et les types de données
- **Statistiques descriptifs :** Aperçu initial de la distribution de données
- **Identification des valeurs manquantes/ outliers et duplicata**
- **Visualisation des distributions**
- **Etude de relation entre les attributs :** Analyse de corrélation



Data Preprocessing

Prétraitement des données

- Nettoyage des données
- Transformation de données
- Discrétisation des données
- Réduction des données

Exploration initiale

DataFrame df=

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden
4	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
271111	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
271112	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271113	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271114	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
271115	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

271116 rows × 7 columns

Identification des types de colonnes

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 7 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   ID      271116 non-null  int64
 1   Name    271116 non-null  object
 2   Sex     271116 non-null  object
 3   Age     261642 non-null  float64
 4   Height  210945 non-null  float64
 5   Weight  208241 non-null  float64
 6   Team    271116 non-null  object
dtypes: float64(3), int64(1), object(3)
memory usage: 14.5+ MB
```

Age, Height & Weight comportent des valeurs manquantes!

Statistiques descriptifs

```
df.describe(include="all")
```

	ID	Name	Sex	Age	Height	Weight	Team
count	271116.000000	271116	271116	261642.000000	210945.000000	208241.000000	271116
unique	NaN	134732	2	NaN	NaN	NaN	1184
top	NaN	Robert Tait McKenzie	M	NaN	NaN	NaN	United States
freq	NaN	58	196594	NaN	NaN	NaN	17847
mean	68248.954396	NaN	NaN	25.556898	175.338970	70.702393	NaN
std	39022.286345	NaN	NaN	6.393561	10.518462	14.348020	NaN
min	1.000000	NaN	NaN	10.000000	127.000000	25.000000	NaN
25%	34643.000000	NaN	NaN	21.000000	168.000000	60.000000	NaN
50%	68205.000000	NaN	NaN	24.000000	175.000000	70.000000	NaN
75%	102097.250000	NaN	NaN	28.000000	183.000000	79.000000	NaN
max	135571.000000	NaN	NaN	97.000000	226.000000	214.000000	NaN

Plus fréquente

Discrets!!

Continus!!

Identification des valeurs manquantes NaN

```
df.isnull().sum()
```

Équivalent à `df.isna().sum()`

```
ID              0
Name            0
Sex             0
Age          9474
Height      60171
Weight      62875
Team            0
dtype: int64
```

Nombre total de valeurs manquantes

```
df.isnull().sum().sum()
#9474+60171+62875=132520
```

Valeurs manquantes

df=

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden
4	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
271111	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
271112	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271113	135570	Piotr ya	M	27.0	176.0	59.0	Poland
271114	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
271115	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

Nettoyage des données : Traitement des NaN



Suppression des instances/ colonnes ayant des valeurs manquantes

`dropna`



Remplissage des valeurs manquantes

Par une valeur fictive

`fillna`
`bfill`
`ffill`

Par l'élément le plus fréquent/ moyenne/médiane

`SimpleImputer`



Traitement des NaN Suppression



```
df.dropna(subset=["Height","Weight"],
          axis=0, inplace=True,
          ignore_index=True)
```

Valeur par défaut

df=

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	5	Christine Jacoba Aaftink	F	21.0	185.0	82.0	Netherlands
3	5	Christine Jacoba Aaftink	F	25.0	185.0	82.0	Netherlands
4	5	Christine Jacoba Aaftink	F	27.0	185.0	82.0	Netherlands
...	...	...	...	...	...	...	...
146744	135568	Olga Igorevna Zyuzkova	F	33.0	171.0	69.0	Belarus
146745	135569	Andrzej ya	M	29.0	179.0	89.0	Poland-1
146746	135570	Piotr ya	M	27.0	176.0	59.0	Poland
146747	135571	Tomasz Ireneusz ya	M	30.0	185.0	96.0	Poland
146748	135571	Tomasz Ireneusz ya	M	34.0	185.0	96.0	Poland

```
df.dropna(subset="Age",axis=1, inplace=True)
```

Conserver uniquement les lignes/colonnes avec au moins 2 valeurs non manquantes!

```
df.dropna(thresh=2, how="any", inplace=True)
```

```
df.dropna(thresh=2, how="any", inplace=True)
```

- any" : Supprimer la ligne ou la colonne si au moins une valeur est NaN
- all" : Supprimer la ligne ou la colonne uniquement si toutes les valeurs sont NaN

Traitement des NaN Remplissage (1/2)

Par une valeur fictive

Nombre de NAN total à remplir!

```
df['Age'].fillna(0, inplace=True ,limit=2)

df['Age'].fillna(df[['Height', 'Weight']].apply(
            lambda x: 40 if (x.iloc[0] >= 170 and x.iloc[1] >= 80) else 25,
                                axis=1), inplace=True)
```

df=

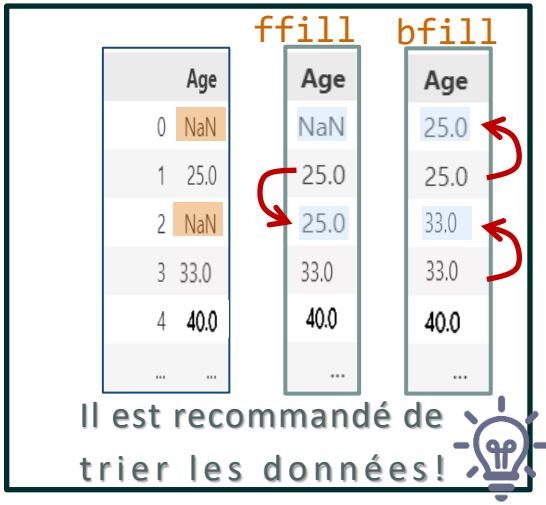
	Age	Height	Weight	Age	Age
0	NaN	180.0	80.0	0.0	40.0
1	NaN	170.0	60.0	0.0	25.0
2	NaN	185.0	82.0	NaN	40.0
3	NaN	185.0	82.0	NaN	40.0
4	NaN	185.0	82.0	NaN	40.0
...	...	...	...	...	...

Remplissage vers l'avant

```
df['Age'].ffill(inplace=True)
df['Age'].fillna(method='ffill', inplace=True)
```

Remplissage vers l'arrière

```
df['Age'].bfill(inplace=True)
df['Age'].fillna(method='bfill', inplace=True)
```



Traitement des NaN Remplissage (2/2)

Par l'élément le plus fréquent/ moyenne/médiane

```
from sklearn.impute import SimpleImputer
mf_imputer= SimpleImputer(missing_values=np.nan, strategy='most_frequent')
arr=mf_imputer.fit_transform(df.loc[:,['Age']])
ndarray

pd.DataFrame(arr,columns=['Age'])
```

Il faut avoir un DataFrame ou ndarry 2D!

Age
24
24
NaN
25
30

'most_frequent'

Age
24.0
24.0
24.0
25.0
30.0

`strategy='mean'` / `strategy='median'`

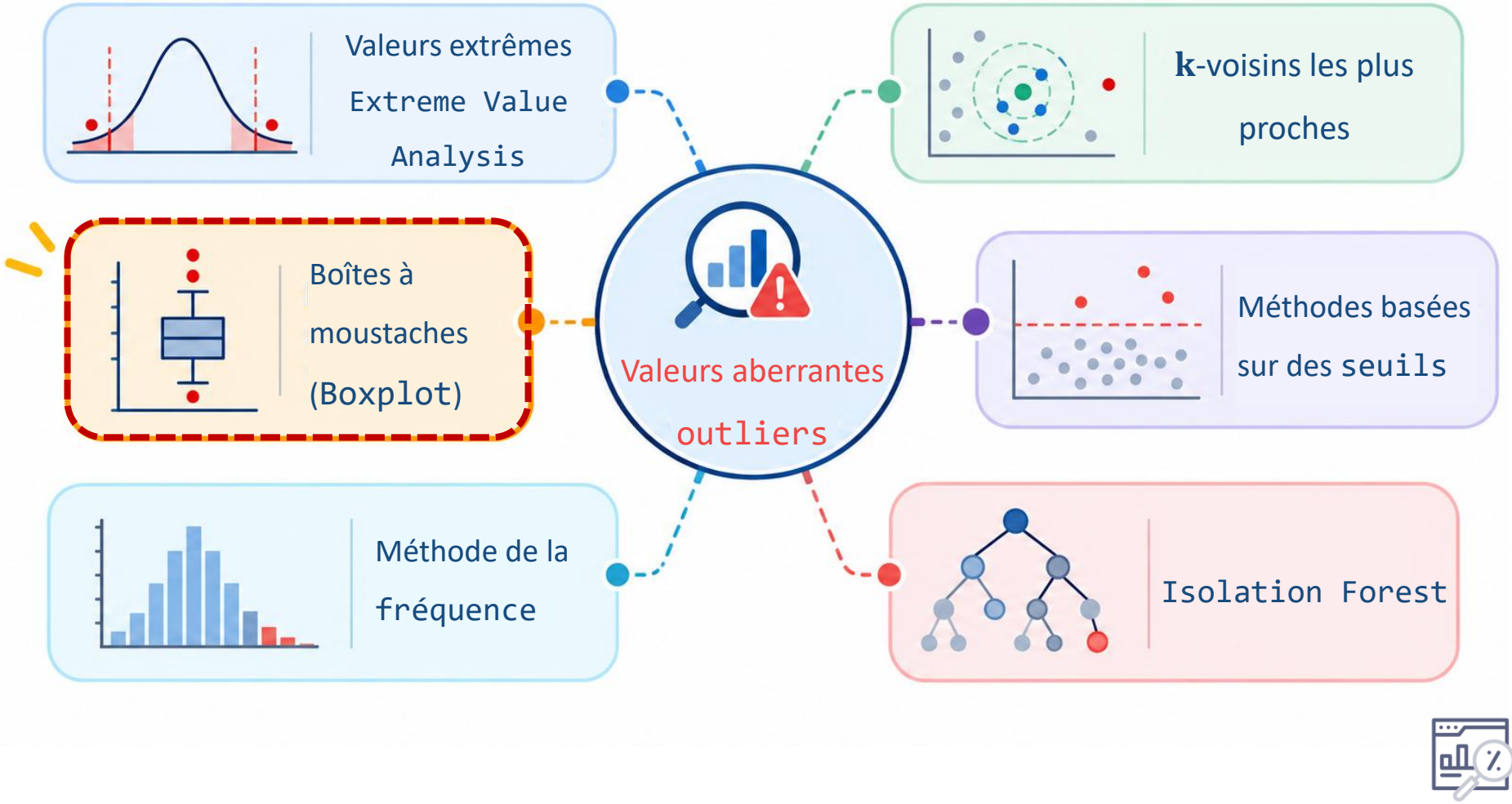
Toutes les valeurs des attributs doivent être de type continue



Traitement des NaN Recap

Situation	Méthode recommandée
Variable numérique, distribution symétrique	`mean`
Variable numérique avec outliers ou forte asymétrie	`median`
Variable catégorielle	`most_frequent (mode)`
Série temporelle, dernière valeur pertinente	`ffill`
Série temporelle, prochaine valeur pertinente connue	`bfill`
Mesures continues avec évolution régulière	Interpolation(remplacer une valeur manquante (NaN) en estimant sa valeur à partir des observations voisines)
Variables numériques corrélées	`KNNImputer`
Relations multivariées complexes	`IterativeImputer`
Valeur manquante ayant une signification métier	Constante adaptée

Identification des valeurs aberrantes

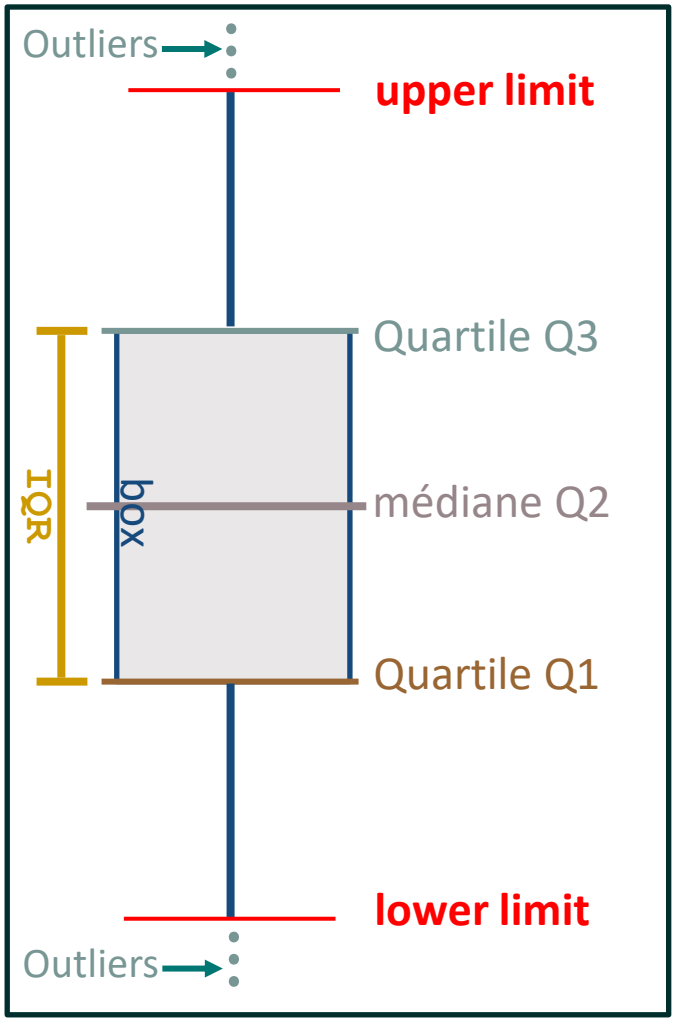


Identification des valeurs aberrantes Boxplot

- **Premier quartile Q1**= 25e percentile= **25%** des données sont inférieures aux valeurs de **Q1**
- **Deuxième quartile Q3**= 75e percentile= **75%** des données sont inférieures aux valeurs de **Q3**
- **Médiane** = **Q2**= 50e percentile
- **Interquartile range (IQR) =Q3−Q1**

lower limit= max (min, Q1−1.5 × IQR)

upper limit= min (max, Q3+1.5 × IQR)



Boxplot Q1, Q3 ?

Soit la distribution de données T suivante : **N=8 observations**

Valeur	10	15	20	25	30	35	40	45
Position	0	1	2	3	4	5	6	7

```
Q1, Q3= np.percentile(T, [25,75])
```

$\text{Index_Q1}=(\frac{25}{100}) \times (N-1)=1.75$

$\text{Index_Q3}=(\frac{75}{100}) \times (N-1)=5.25$

Q1 =15+(20-15)× partie décimale(Index_Q1)= 15+(20-15)×0.75=18.75

Q3 =35+(40-35)× partie décimale(Index_Q3)= 35+(40-35)×0.25=36.25

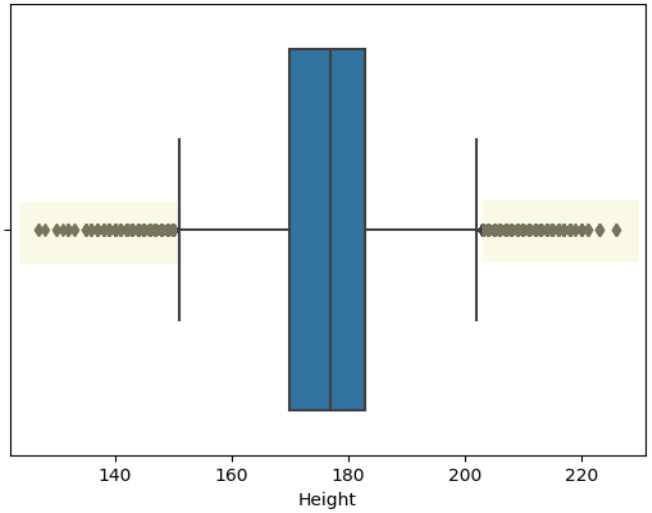
Nettoyage des données : Traitement des outliers

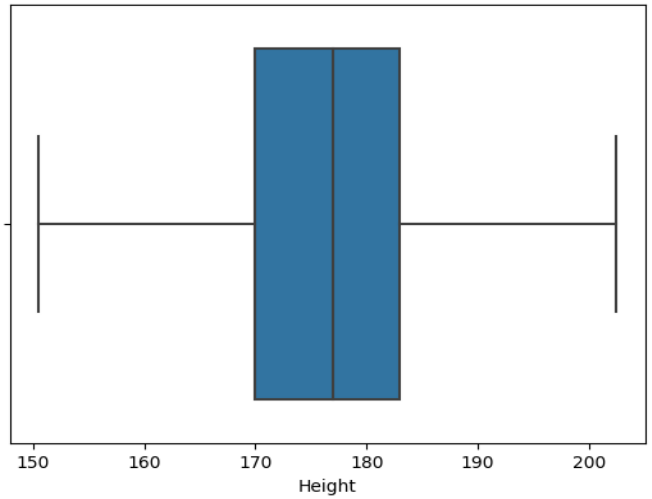
```
sns.boxplot(data=df,x="Height")

Q1,Q3=np.percentile(df["Height"],[25,75])
IQR=Q3-Q1
upper_limit=Q3+1.5*IQR
lower_limit=Q1-1.5*IQR

outliers=df["Height"][(df["Height"]>upper_limit)|
                      (df["Height"]<lower_limit)]
```

```
df['Height']=np.where(df['Height']>=upper_limit,
    upper_limit, np.where(df['Height']<=lower_limit,
        lower_limit,df['Height']))
```





Identification & Traitement des lignes dupliquées

- **Détection des lignes dupliquées**

```
df1=df[df.duplicated()]
```

df=

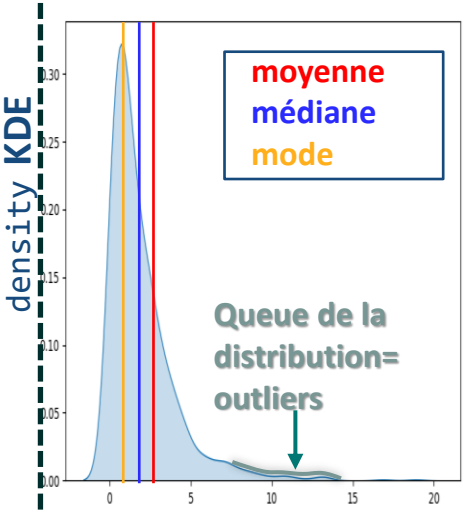
	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	1	A Dijiang	M	24.0	180.0	80.0	China
2	2	A Lamusi	M	23.0	170.0	60.0	China
3	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
4	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden

- **Suppression des lignes dupliquées**

```
df.drop_duplicates(inplace=True,
                   ignore_index=True)
```

	ID	Name	Sex	Age	Height	Weight	Team
0	1	A Dijiang	M	24.0	180.0	80.0	China
1	2	A Lamusi	M	23.0	170.0	60.0	China
2	3	Gunnar Nielsen Aaby	M	24.0	NaN	NaN	Denmark
3	4	Edgar Lindenau Aabye	M	34.0	NaN	NaN	Denmark/Sweden

Type de distribution de données continues

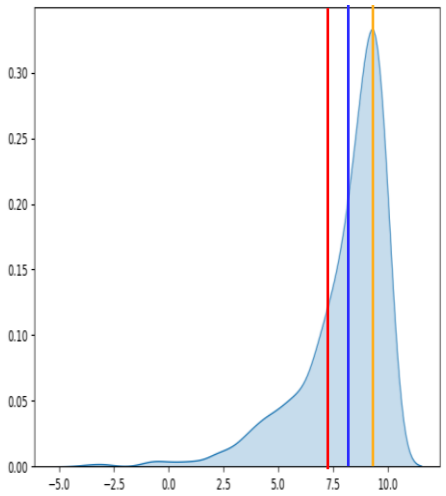


Asymétrique positive=

Asymétrique à droite

(right-skewed)

mode<médiane< moyenne

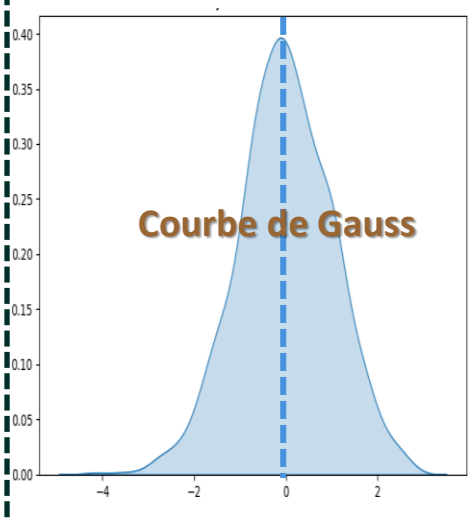


Asymétrique négative=

Asymétrique à gauche

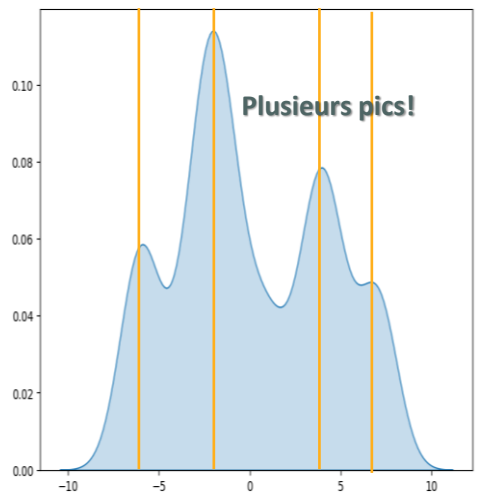
(left-skewed)

moyenne<médiane< mode



Symétrique

moyenne=médiane= mode



Multimodale

(Non-alignement)

Plusieurs modes

Visualisation des distributions Attribut continu

Densité avec une partition gaussienne =Kernel Density Estimator(KDE)

```
sns.kdeplot(df['Weight'],
            bw_method='scott', bw_adjust=1)
```

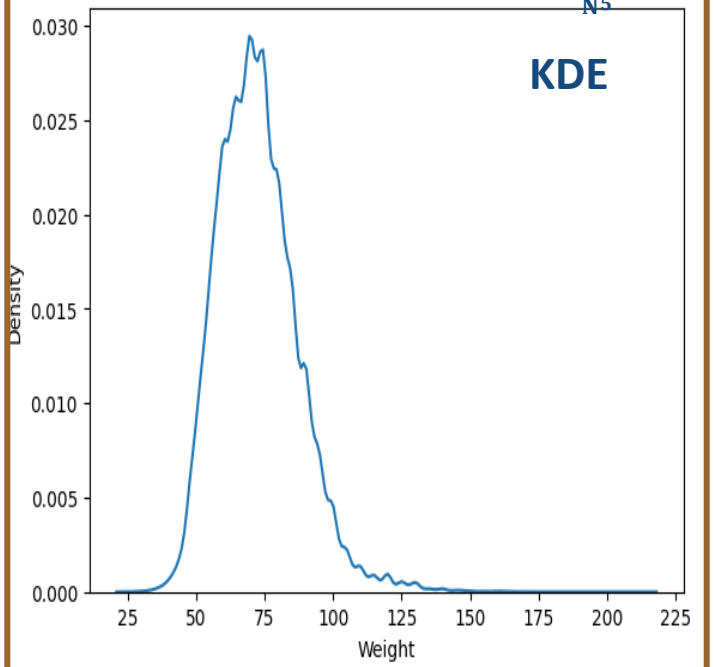
FACTEUR D'AJUSTEMENT

bw_adjust >1: Estimation de densité plus lisse (courbe plus lissée avec moins de détails)

bw_adjust <1: Estimation de densité moins lisse (courbe plus détaillée avec plus de variation)

$$Density(x)=\frac{1}{Nh}\sum_{i=1}^{N} K\left(\frac{x-x_i}{h}\right)$$

$$bw('scott')=h=bw_adjust \times \left(\frac{0.479.std(x)}{\frac{1}{N^{\frac{1}{5}}}}\right)$$

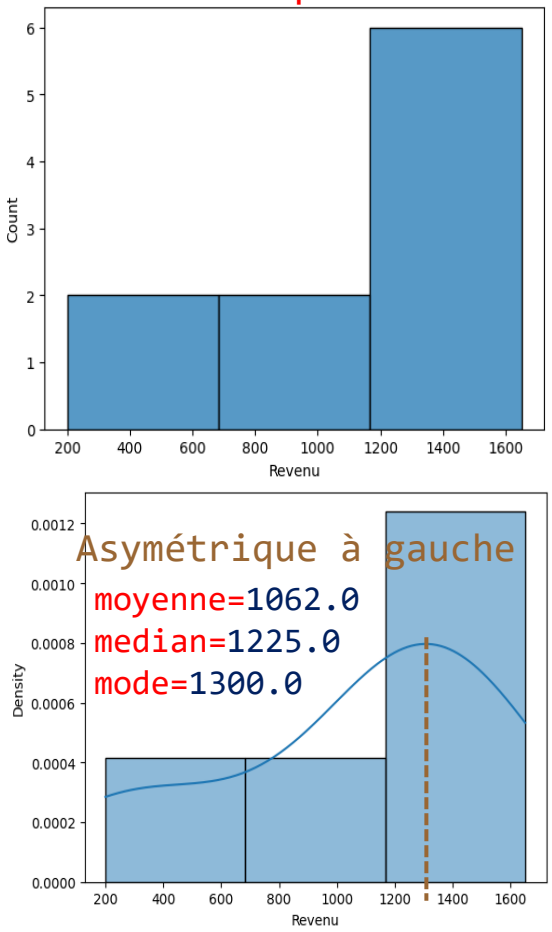


Histogramme de données continues histplot+kde

```
sns.histplot(x="Revenu", data=df,
                         bins=3)
```

```
sns.histplot(x="Revenu", data=df,
             bins=3, kde=True, stat='density')
```

histplot



N=10

df=	Revenu
0	x_1= 300
1	x_2= 1100
2	1200
3	1500
4	200
5	1400
6	1250
7	1300
8	1650
9	720

histplot+kde/ choix optimal de nombre de bins!

- **Règle de Freedman-Diaconis**

  nombre de bins= $2 \times IQR \times N^{(-\frac{1}{3})}$

- **Règle de Scott**

  nombre de bins= $3.5 \times \frac{std(\Omega)}{N^{(\frac{1}{3})}}$

- **Règle de racine carrée**

  nombre de bins=$\sqrt{N}$

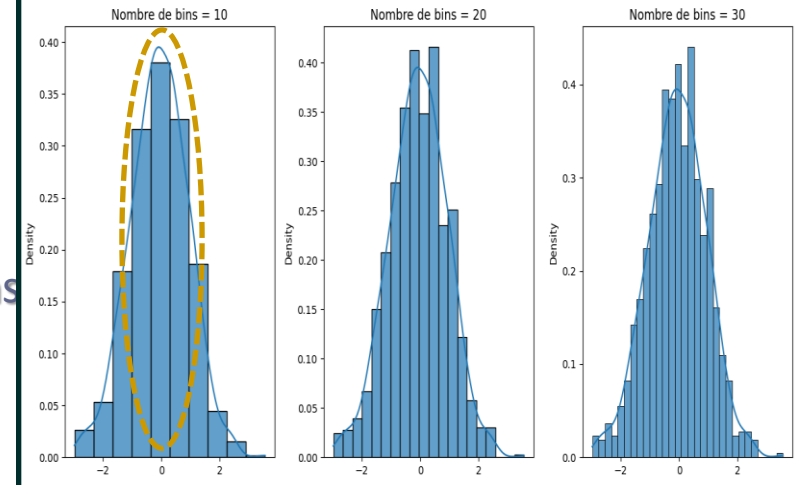
- **Règle de Sturges (optimale pour des échantillons de taille limitée)**

  nombre de bins=$1 + \log_2(N)$

- **Expérimentation :** Essayer plusieurs valeurs de bins & choisir celle qui offre la représentation la plus informative de la distribution des données

Dataset Ω

	X_1	...	X_K
obs1	a_{11}		a_{1K}
⋮	⋮	⋮	⋮
obsi	a_{i1}		a_{iK}
obsN	a_{N1}		a_{NK}



Un choix judicieux du nombre de bins peut donner un histogramme qui reflète bien la distribution!

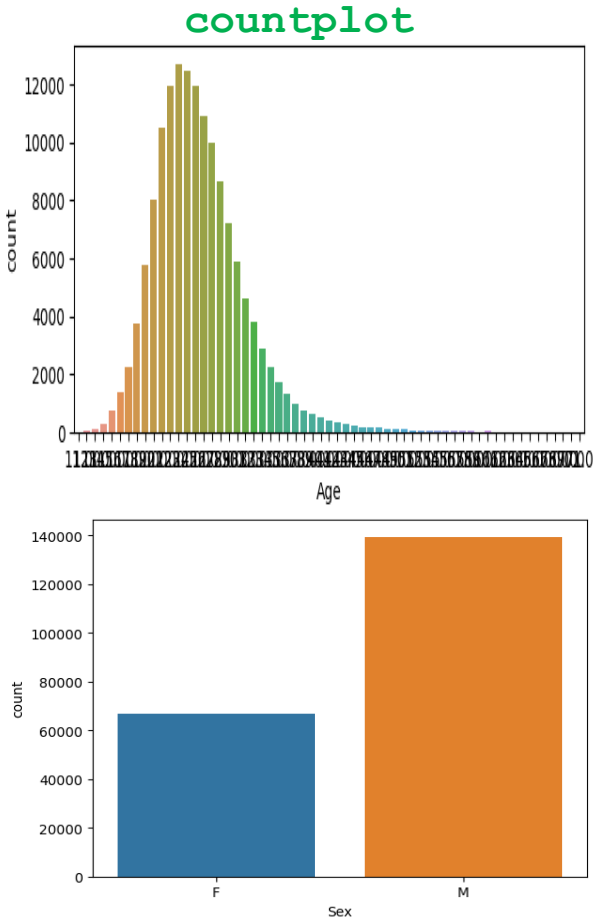
Histogramme de données discrètes countplot

Attribut continu

```
sns.countplot(x="Age", data=df, bins=3)
```

Attribut discret

```
sns.countplot(x="Sex", data=df ,
                        order=["F","M"])
```



df=	Sex	Age
0	M	24.0
1	M	23.0
4	F	21.0
5	F	21.0
6	F	25.0
...	...	...
271111	M	29.0
271112	M	27.0
271113	M	27.0
271114	M	30.0
271115	M	34.0

Transformation des données *numériques* 1/4

Normalisation Min − Max

Transformation des valeurs de caractéristiques à l'échelle pour qu'elles se situent entre un maximum et un minimum

$$a_{ji} = \frac{a_{ji} - \min(x_i)}{\max(x_i) - \min(x_i)} \quad [0 \dots 1]$$

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler=MinMaxScaler()
arr=min_max_scaler.fit_transform(df)
pd.DataFrame(arr,columns=['X1','X2'])
```

	x_1	...	x_i	x_p
obs1	a_{11}	⋮	a_{1i}	a_{1p}
⋮	⋮	⋮	⋮	⋮
obsj	a_{j1}	⋮	a_{ji}	a_{jp}
obsN	a_{N1}	...	a_{Ni}	a_{Np}

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après MinMaxScaler

	X1	X2
0	1.000000	0.555556
1	0.000000	1.000000
2	0.571429	0.000000

☺ **Optimum pour une distribution uniforme**

☺ **Préserve la forme de la distribution**

☹ **Sensibles aux valeurs aberrantes**

Transformation des données *numériques* 2/4

Standardization zscore z_{ji}

Transformation des données en divisant la différence entre les données a_{ji} et la moyenne M_i par l'écart type σ_i $z_{ji} = \frac{a_{ji} - M_i}{\sigma_i}$ $]-\infty \dots +\infty[$

avec $M_i = \frac{1}{N}\sum_{j=1}^{N} a_{ji}$ et $\sigma_i = \sqrt{\frac{\sum_{j=1}^{N}(a_{ji}-M_i)^2}{N}}$ et N= Nombre d'observations

```
from sklearn.preprocessing import StandardScaler
standard_scaler= StandardScaler()
arr= standard_scaler.fit_transform(df)
pd.DataFrame(arr,columns=['X1','X2'])
```

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après MinMaxScaler

	X1	X2
0	1.162476	0.090536
1	-1.278724	1.176965
2	0.116248	-1.267500

☺ **Optimum pour une distribution symétrique**

☺ **Préserve la forme de la distribution**

☹ **Sensibles aux valeurs aberrantes**

Transformation des données *numériques* 3/4

Standardization RobustScaler

Transformation des données en utilisant la médiane et l'écart interquartile (**IQR**)

$$z_{ji} = \frac{a_{ji} - Q2}{IQR} \quad]-\infty \dots +\infty[$$

avec **IQR**= **Q3** - **Q1** ; **Q1**= 25e percentile ; **Q3**= 75e percentile

et **Q2**= 50e percentile= médiane

```
from sklearn.preprocessing import RobustScaler
standard_scaler= RobustScaler()
arr= standard_scaler.fit_transform(df)
pd.DataFrame(arr,columns=['X1','X2'])
```

df=

	X1	X2
0	25	40
1	18	44
2	22	35

Après RobustScaler

	X1	X2
0	0.857143	0.000000
1	-1.142857	0.888889
2	0.000000	-1.111111

☺ **Optimum pour une distribution asymétrique**

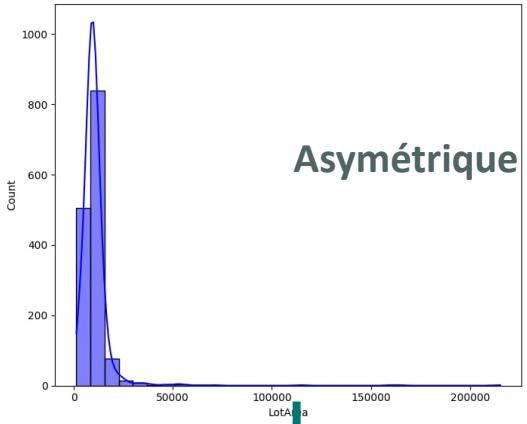
☺ **Préserve la forme de la distribution**

☺ ***Optimum lorsqu'il y a des valeurs aberrantes dans les données***

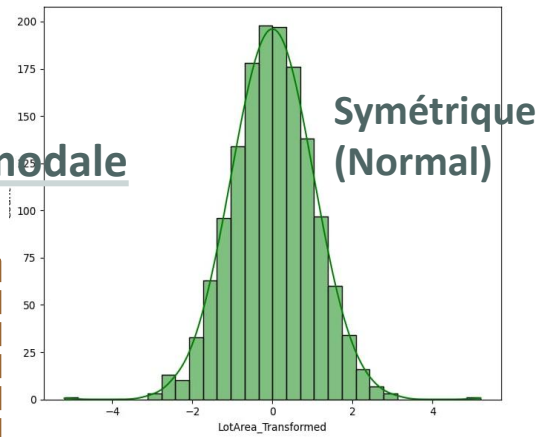
Transformation des données *numériques* 4/4

QuantileTransformer

- Mappe les données sur une distribution uniforme ou gaussienne, en tenant compte de la forme de la distribution d'origine




```
from sklearn.preprocessing import QuantileTransformer
transformer = QuantileTransformer(output_distribution='normal')  #ou 'uniform'
arr = transformer.fit_transform(df)
```



☺ **Optimum pour une distribution asymétrique ou multimodale**

Distribution multimodale

La transformation ajuste toutes les parties de la distribution, ce qui peut réduire l'impact des modes, mais ne garantit pas la symétrie

Transformation des données *discrètes (Encoding)* 1/3

OrdinalEncoder & LabelEncoder 1/2

—Convertir des chaines de caractères en nombres entiers ordinaux

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder()
df[["Sex","Season"]]=enc.fit_transform(
                    df.loc[:,["Sex", "Season"]])
```

```
from sklearn.preprocessing import LabelEncoder
enc = LabelEncoder()
df[["Sex","Season"]]=enc.fit_transform(
                    df.loc[:,["Sex", "Season"]])
df["Sex"]=enc.fit_transform(df.loc[:,"Sex"])
```

Série 1D

	Sex	Season
0	M	Summer
1	M	Summer
2	M	Summer
3	M	Summer
4	F	Springer
5	F	Autumn
6	F	Autumn
7	F	Springer
8	F	Winter
9	F	Winter
10	M	Summer
11	M	Autumn
12	M	Springer

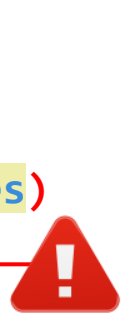
	Sex	Season
0	1.0	2.0
1	1.0	2.0
2	1.0	2.0
3	1.0	2.0
4	0.0	1.0
5	0.0	0.0
6	0.0	0.0
7	0.0	1.0
8	0.0	3.0
9	0.0	3.0
10	1.0	2.0
11	1.0	0.0
12	1.0	1.0

Transformation des données *discrètes (Encoding)* 2/3

OrdinalEncoder / LabelEncoder 2/2

Comment spécifie-t-on l'ordre des catégories lors de l'utilisation de `OrdinalEncoder` ou `LabelEncoder`?!

```
from sklearn.preprocessing import OrdinalEncoder,
                                  Label Encoder
ord_categories= [["M", "F"],
   ["Summer", "Springer", "Autumn", "Winter"]]
enc1 = OrdinalEncoder(categories=ord_categories)
enc2 = LabelEncoder(categories=["M", "F"])
df[["Sex","Season"]]=enc1.fit_transform(
                    df.loc[:,["Sex", "Season"]])
```



	Sex	Season
0	M	Summer
1	M	Summer
2	M	Summer
3	M	Summer
4	F	Springer
5	F	Autumn
6	F	Autumn
7	F	Springer
8	F	Winter
9	F	Winter
10	M	Summer
11	M	Autumn
12	M	Springer

	Sex	Season
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	1.0	1.0
5	1.0	2.0
6	1.0	2.0
7	1.0	1.0
8	1.0	3.0
9	1.0	3.0
10	0.0	0.0
11	0.0	2.0
12	0.0	1.0

Transformation des données *discrètes (Encoding)* 3/3

OneHotEncoder/ get_dummies()

- Transformer l'espace de **n** descripteurs à **k** descripteurs binaires (**k** > **n**)

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False,
                    categories=[["M", "F"]])
df[["S=M","S=F"]]=enc.fit_transform(
                        df.loc[:,["Sex"]])
```

DataFrame 2D!

```
df.drop(labels=["Sex"], axis=1, inplace=True)
```

```
df=pd.get_dummies(df, columns=['Sex'],
  prefix='Gender', prefix_sep='_',dtype='int32')
```

	Sex	Season
0	M	Summer
1	M	Summer
2	M	Summer
3	M	Summer
4	F	Springer
5	F	Autumn
6	F	Autumn
7	F	Springer
8	F	Winter
9	F	Winter
10	M	Summer
11	M	Autumn
12	M	Springer

	Season	S=M	S=F
0	Summer	1.0	0.0
1	Summer	1.0	0.0
2	Summer	1.0	0.0
3	Summer	1.0	0.0
4	Springer	0.0	1.0
5	Autumn	0.0	1.0
6	Autumn	0.0	1.0
7	Springer	0.0	1.0
8	Winter	0.0	1.0
9	Winter	0.0	1.0
10	Summer	1.0	0.0
11	Autumn	1.0	0.0
12	Springer	1.0	0.0

Discrétisation des données Méthodes



Equal-Width Discretization

`'uniform'`



Equal-Frequency Discretization

`'quantile'`



K-Means Discretization

`'K-Means'`

NB.

- Il n'y a pas une discrétisation optimale
- Il y a une démarche cohérente par rapport à un objectif donné et une distribution donnée

Discrétisation des données 1/2

Equal-Width Discretization 'uniform'

- Séparer toutes les valeurs possibles de chaque descripteur en n_bins intervalles (classes), chacune ayant la même largeur

$$\text{Width}=\frac{\max(x_i)-\min(x_i)}{\text{n_bins}}$$

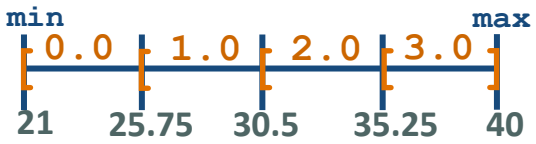
```
from sklearn.preprocessing import KBinsDiscretizer
enc = KBinsDiscretizer(n_bins=4, encode='ordinal',
                       strategy='uniform', subsample=None)
df[['Age']] = enc.fit_transform(df[['Age']])
```

	Age
0	24.0
1	23.0
2	24.0
3	34.0
4	21.0
5	22.0
6	25.0
7	40.0
8	27.0
9	38.0
10	31.0

	Age
0	0.0
1	0.0
2	0.0
3	2.0
4	0.0
5	0.0
6	0.0
7	3.0
8	1.0
9	3.0
10	2.0

$$\frac{40-21}{4}=4.75$$

☺ **Optimum pour une distribution uniforme**



Discrétisation des données 2/2

Equal-Frequency Discretization 'quantile'

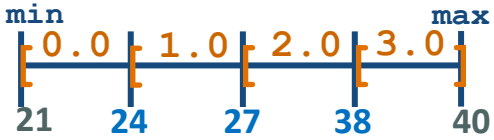
- Séparer toutes les valeurs possibles de chaque descripteur en n_bins intervalles (classes), chacune ayant ± le même nombre d'observation

```
from sklearn.preprocessing import KBinsDiscretizer
enc = KBinsDiscretizer(n_bins=4, encode='ordinal',
                       strategy='quantile', subsample=len(df))
df[['Age']] = enc.fit_transform(df[['Age']])
```

	Age
0	24.0
1	23.0
2	24.0
3	34.0
4	21.0
5	22.0
6	25.0
7	40.0
8	27.0
9	38.0
10	31.0

	Age
0	1.0
1	0.0
2	1.0
3	3.0
4	0.0
5	0.0
6	2.0
7	3.0
8	2.0
9	3.0
10	2.0

21	22	23	24	24	25	27	31	34	38	40
0	1	2	3	4	5	6	7	8	9	10



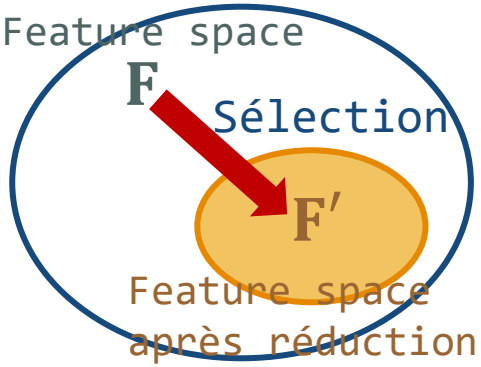
$$Index_i = \frac{i \times n}{n_bins} \quad Index_1 = \frac{1 \times 11}{4} = 2.75 \cong 3 \quad Index_2 = \frac{2 \times 11}{4} = 5.5 \cong 6 \quad Index_3 = \frac{3 \times 11}{4} = 8.25 \cong 9$$

☺ Optimum pour une distribution symétrique et asymétrique

Réduction des données

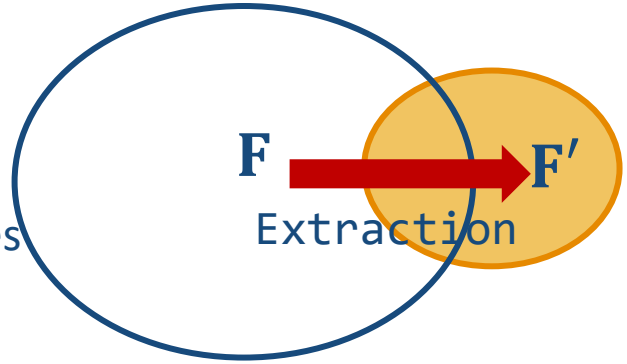
Sélection de descripteurs

- Sélection des descripteurs les plus pertinentes à partir de l'ensemble initial de caractéristiques



Extraction de descripteurs

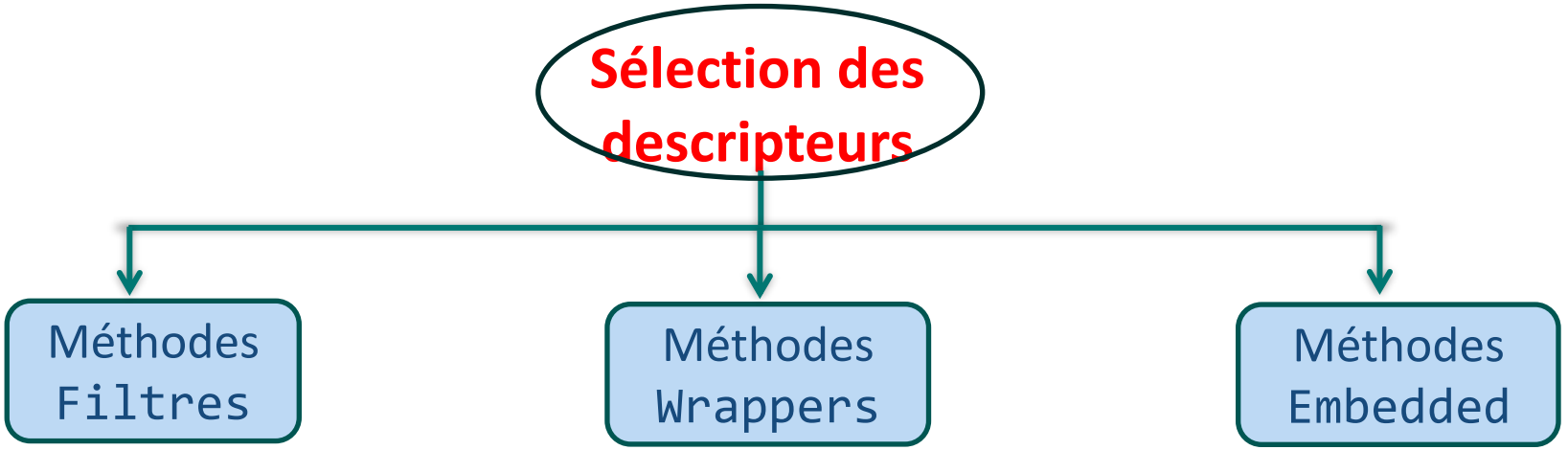
- Réduction basée sur une transformation des données
- Construire un nouvel ensemble réduit de descripteurs à partir de l'ensemble initial



F : feature space

F′: feature space après réduction

Réduction des données Sélection des descripteurs



- Corrélation
- Variance
- Dispersion ratio
- Information gain

- Forword Feature Selection
- Backward Feature selection

- Regularisation L1,L2
- Random Forest importance

Analyse de corrélation 1/2 Pearson

Mesure la relation linéaire entre deux variables

$$Corr(X,Y)=\frac{\sum_{i=1}^{N}(x_i-\bar{X})(y_i-\bar{Y})}{\sqrt{\sum_{i=1}^{N}(x_i-\bar{X})^2\sum_{i=1}^{N}(y_i-\bar{Y})^2}} \qquad [-1..1]$$

$\bar{X}=\frac{\sum_{i=1}^{N}x_i}{N}$ = moyenne de X

$\bar{Y}=\frac{\sum_{i=1}^{N}x_i}{N}$ = moyenne de Y

avec N= Nombre d'observations

X	Y
x_1	y_1
⋮	⋮
x_i	y_i
⋮	⋮
x_n	y_n

df=

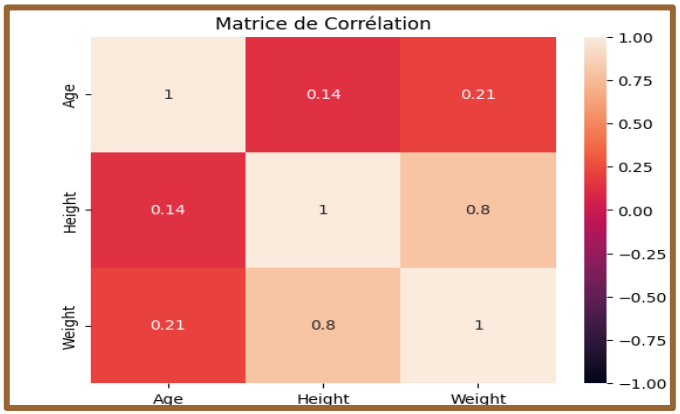
	Age	Height	Weight
0	24.0	180.0	80.0
1	23.0	170.0	60.0
2	24.0	NaN	NaN
3	34.0	NaN	NaN
4	21.0	185.0	82.0
...	...	...	...
271111	29.0	179.0	89.0
271112	27.0	176.0	59.0
271113	27.0	176.0	59.0
271114	30.0	185.0	96.0
271115	34.0	185.0	96.0

df_corr=

	Age	Height	Weight
Age	1.000000	0.138246	0.212069
Height	0.138246	1.000000	0.796213
Weight	0.212069	0.796213	1.000000

par défaut `numeric_only=False`

```
df_corr=df.corr(numeric_only= True)
sns.heatmap(df_corr,annot=True,vmin=-1, vmax=1)
plt.title('Matrice de Corrélation')
plt.show()
```

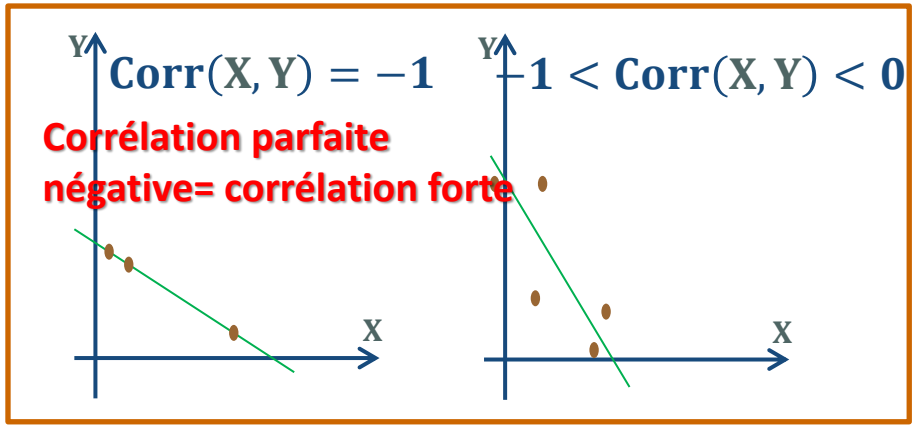


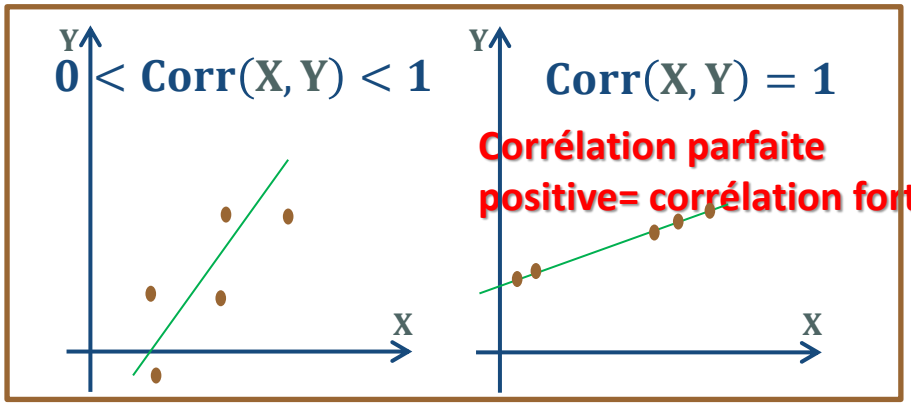
Analyse de corrélation 2/2 Pearson

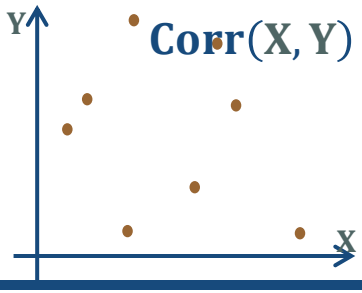
☑ **Corrélation négative=** X et Y évoluent ensemble de manière linéaire dans des directions opposées!

☑ **Corrélation Positive=** X et Y évoluent ensemble de manière linéaire dans la même direction!

☑ **Corrélation=0 = Absence de corrélation linéaire entre les attributs X et Y !**







Analyse de corrélation Autres techniques

Technique	Variables	Mesure / utilité	Valeurs possibles	Interprétation
Spearman (ρ)	Numérique / ordinale ↔ numérique / ordinale	Corrélation monotone à partir des rangs	−1 à +1	ρ ≈ +1 : croissante ρ ≈ −1 : décroissante ρ ≈ 0 : pas de relation monotone
Chi² (χ^2)	Catégorielle ↔ catégorielle	Test d'indépendance entre catégories	$\chi^2 \geq 0$ p : 0 à 1	p < 0,05 : association statistiquement significative
ANOVA (F)	Catégorielle ↔ numérique	Test des différences de moyennes	F ≥ 0 p : 0 à 1	p < 0,05 : au moins deux moyennes diffèrent
Information mutuelle	Variables diverses (estimateur adapté)	Dépendance, même non linéaire	MI ≥ 0	MI = 0 : indépendance (théorique) Plus élevée = plus de dépendance ; pas de seuil universel
Phi (ϕ)	Binaire ↔ binaire	Sens et intensité de l'association	−1 à +1	ϕ ≈ ±1 : association forte (+ ou −) ϕ ≈ 0 : faible association

χ^2 et F sont des statistiques de test

Analyse de corrélation Recap

Attribut 1	Attribut 2	Méthode conseillée
Numérique	Numérique	Pearson (linéaire), Spearman (monotone)
Numérique	Binaire	Point-bisérial (équivalent à Pearson avec codage 0/1)
Numérique	Catégorielle nominale	η^2, ANOVA
Numérique	Ordinale	Spearman
Catégorielle nominale	Catégorielle nominale	Chi^2, information mutuelle
Binaire	Binaire	Chi^2
Binaire	Catégorielle nominale	Chi^2
Ordinale	Ordinale	Spearman
Ordinale	Catégorielle nominale	Chi^2
Binaire	Ordinale	Spearman (si l'ordre est pertinent)

Variance VarianceThreshold

$$var(X_j) = \frac{1}{N}\sum_{i=1}^{N}(a_{ij} - \bar{X}_j)^2 \qquad [0, +\infty[$$

$$\bar{X}_j = \frac{1}{N}\sum_{i=1}^{N} a_{ij}$$

	X_j
obs1	a_{1j}
⋮	⋮
obsi	a_{ij}
obsN	a_{Nj}

Tous les attributs doivent être de type numérique + Pas de NaN



df=

	Sex	Age	Height	Weight	Year	City	Sport
0	M	24.0	180.0	80.0	1992	Barcelona	Basketball
1	M	23.0	170.0	60.0	2012	London	Judo
4	F	21.0	185.0	82.0	1988	Calgary	Speed Skating
5	F	21.0	185.0	82.0	1988	Calgary	Speed Skating
6	F	25.0	185.0	82.0	1992	Albertville	Speed Skating
...	...	...	...	...	...	...	...
271111	M	29.0	179.0	89.0	1976	Innsbruck	Luge
271112	M	27.0	176.0	59.0	2014	Sochi	Ski Jumping
271113	M	27.0	176.0	59.0	2014	Sochi	Ski Jumping
271114	M	30.0	185.0	96.0	1998	Nagano	Bobsleigh
271115	M	34.0	185.0	96.0	2002	Salt Lake City	Bobsleigh

```
from sklearn.feature_selection import VarianceThreshold
```

Par défaut=0

```
thresholder = VarianceThreshold(threshold=40)
```

Conserver X_j si $Var(X_j) > threshold$

`threshold=0.0` : la méthode supprime les features constantes

```
arr= thresholder.fit_transform(df.loc[:,["Age","Height","Weight","Year"]])
```

Réduction des données Extraction des descripteurs

- PCA (Principal Component Analysis)
- LDA (Linear Discriminant Analysis)

Méthodes linéaires

- SVD (Singular value Decomposition)
- Kernel PCA
- T-SNE (t-Distributed Stochastic Neighbor Embedding)
- Auto-encodeurs (Autoencoders)

Méthodes non-linéaires



Tests statistiques & Data Preprocessing 1/2

Pourquoi utiliser des tests statistiques ?

Explorer
Observer distributions, groupes et relations

→

Questionner
Normalité ? Variances ? Association ?

→

Effectuer un test
Obtenir une statistique & une p-value

→

Décider et valider
Choisir une approche, puis vérifier son utilité

Trois usages concrets

Distribution & normalité

`Shapiro-Wilk`

Vérifier si une hypothèse de normalité est plausible avant certaines analyses statistiques.

Comparaison de groupes

`Levene`

Repérer des variances inégales & choisir un test de comparaison adapté.

Lien avec la cible

`Chi² · ANOVA`

Identifier des features potentiellement associées à la classe ou à une variable cible.



Tests statistiques & Data Preprocessing 2/2

TEST	QUESTION À POSER	SI p < 0,05…	UTILITÉ EN PREPROCESSING
Shapiro–Wilk	La variable (ou les résidus) suit-elle une loi normale ?	**Écart à la normalité**	Évaluer une hypothèse de normalité lorsqu'elle est nécessaire.
Levene	Les variances de plusieurs groupes sont-elles égales ?	**Variances différentes**	vérifier les hypothèses des analyses statistiques
Chi²	Deux variables catégorielles sont-elles indépendantes ?	**Association significative**	Étudier feature ↔ classe
ANOVA (F)	Les moyennes sont-elles égales entre les groupes ?	**Au moins un groupe diffère**	Étudier une feature numérique selon les classes ; compléter par η^2.
Mann–Whitney / Kruskal–Wallis	Les distributions des groupes sont-elles identiques ?	**Différence entre groupes.**	Alternatives fondées sur les rangs, selon le contexte.

Lire la p-value (seuil indicatif : 5 %)

$p < 0,05$: rejeter H_0 · $p \geq 0,05$: ne pas rejeter H_0

Deux précautions indispensables

Significatif ≠ important. Pour sélectionner les features, tester sur le train uniquement (éviter la fuite de données).